Technologies and Products

Haachtsesteenweg 1442 Chaussée de Haecht
B-1130 Brussels
Belgium

Reference Functional Specifications

VIC Standard

Version: 1.07/13-03 (Final)

© Atos Worldline sa/nv 2013

Copyright Notice

The information contained in this document is subject to change without notice and should not be construed as a commitment by **Atos Worldline**.

Atos Worldline assumes no responsibility for any errors or omissions that may appear in this document.

The contents of this document must not be reproduced in any form whatsoever, by or on behalf of third parties, without the prior written consent of **Atos Worldline**

Disclaimer

It is the Customer's exclusive responsibility to take all appropriate security measures pertaining to the operation of the **Atos Worldline** product. **Atos Worldline** disclaims any responsibility in that respect.

Procedures defined in these guidelines are designed for information only. The Customer assumes full responsibility for their use and/or selection. **Atos Worldline** does not warrant the result of the use of these procedures, not that the procedures contained herein will meet the Customer's requirements or fitness for any purpose.

1. Document Summary

Ref	:	
Title	:	**VIC Standard**
Version	:	1.07/13-03
Created by	:	T&P
Last modified by	:	MILICHE Damien AWL-T&P
Status	:	Final
Confidentiality	:	
File Path	:	T:\Products\Applications\Product Functionalities\Integration\ECR-VIC\VIC_standard\1-Functional Specifications\vic 1.07\Working versions\VIC 1.0713-03.docx
File Size	:	572 Kb
File version	:	16
Creation date	:	24/04/2013 16:25
Last time edited	:	25/04/2013 10:37
Date Printed	:	25/04/2013 10:36
Number of pages	:	60 Pages
Number of words	:	12626

Version management report

Version	Name(s)	Date	Comments
[…]	[…]	[…]	[…]
V1.07/11	C. Castadot / O. Planckaert	05/10/06	SEPA Migration + Global review
V1.07/11-02	O. Planckaert	25/10/06	Add one bit in the vic_bit_map_application_id for the EZSwitch TPA.
V1.07/11-03	O. Planckaert	28/11/07	Format of the pdv_clct_nr is different for petrol transaction managed by EFT Server
V1.07/12	C. Castadot	27/12/07	Adding Nederland and Luxembourg functionalities.
V1.07/12-02	C. Castadot	30/04/08	Update of the term_id field and the vic_proprietary_data field
V1.07/12-03	C. Castadot	14/08/08	Update iep_tx_incident
V1.07/12-04	C. Castadot	14/08/08	Update iep_tx_incident and vic_data
V1.07/12-05	C. Castadot	22/12/08	Add vic_bitmap_application_id for Pay Fair
V1.07/12-06	C. Castadot	06/05/09	Add vic_bitmap_application_id for Sodexo, Accord and reserved

Version management report

V1.07/12-07	D. Miliche	30/11/11	Add new feature for Contactless transaction Add feature for Power Management Add support for BC/MC EMV
V1.07/13	D.Miliche	18/01/12	Updates Add support for Emergency fallback (TINA EMV) Add feature for Low Value Payment (XENOA) Add support for Partial Approval
V1.07/13-01	D. Miliche	06/04/12	Add support for KADOZ
	D. Miliche	19/07/12	Add support for AWL Partners brand
	D. Miliche	27/09/12	Add support for RES
	J. Vanslembrouck	06/11/12	vic_bit_map_application_id is an all-0 field if card_token is present. A pdv_error is sent in case a token cannot be generated.
	D. Miliche	04/12/12	Add support for Yourgift, Giftcard and ORES
V1.07/13-02	D. Miliche	12/12/12	Externalisation of vic_bit_map_application_id values list to addendum
V1.07/13-02	D. Miliche	24/04/13	Add support for *Cancellation of Unspecified service*

1.1. Release Management

Document version	Changes
[…]	[…]
Version 1.07/11	Add: • vic_version • brand_name in pdv_debit_result • a vic_bitmap_application_id for Visa Vpay • Tags in Function_cc • Hardware type for Xenteo Change: • Keep only one form with tag for vic_data + a new tag for authorization code Remove • Vic_private_id, vic_private_data and recovery_data. • vic_cmd_code from vmc_debit_request. • All code error 6*** • ed_applic_info_request / cz_applic_info_reply • all functionalities related to Full CCM

	Global review
Version 1.07/11-02	Add one bit in the vic_bit_map_application_id for the EZSwitch TPA.
Version 1.07/11-03	Format of the pdv_clct_nr is different for petrol transaction managed by EFT Server
Version 1.07/12	
Version 1.07/12-02	Update of the term_id field and the vic_proprietary_data field and Messages contents summary
Version 1.07/12-03	Update of the iep_tx_incident
Version 1.07/12-04	Update of the iep_tx_incident and add tag for additional data in vic_data
Version 1.07/12-05	Add vic_bitmap_application_id for Pay Fair
Version 1.07/12-06	Add vic_bitmap_application_id for Sodexo, Accord and 2 reserved
Version 1.07/12-07	Adaptation for Contactless transaction: - Add the field Amount in the VMC_purse_request. - Create the new field POS entry mode. - Add the new field POS entry mode in the PDV_purse_result Adaptation for Power Management support: - pdv_state_mask updated with: bit 3 Purchase Device wake up supported to be used with the vmc_command - pdv_state updated with: - bit 3 Purchase Device wake up date/time changed to be used with pdv_state_info (unsolicited message) - date_and_time field added as optional into pdv_state_info (unsolicited message) Adaptation for BC/MC EMV support: - new vic_bit_map_application_id value
Version 1.07/13	Adaptation for Emergency fallback (TINA EMV) support: - add new value for Emergency fallback (TINA EMV) in vic_bit_map_application_id - add new value for the Emergency fallback (TINA EMV) in the transaction_protocol field - new generic formulation for incident codes (5016, 5018, 5019) - new field tsc_id, optional in pdv_debit_result Adaptation for Low Value Payment (XENOA): - add new field card_token for tokenization to be used in pdv_purse_result and pdv_debit_result - add new field additional_info_bit_map, to be used in *vmc_purse_request* and *vmc_debit_request*, in order to specify which information is expected in the response - new incident code (5030, Card information not available) to be transmitted e.g. when the token cannot be generated - add new conditions for sending a *vmc_cancel* - add new bit to pdv_state and pdv_state_mask for the configuration and indication of MMI synchronizations - add new field display_text_ref to be used in *pdv_debit_result* and *pdv_state_info* - *vic_version* optional in pdv_error for the startup - vic_config_clct_amt, curcy and vic_bit_map_application_id become optional in a configuration *vmc_command*, in order to activate synchronization independently of the (Proton) application - vic_bit_map_application_id becomes optional in a configuration *pdv_command_reply* Adaptation for Partial Approval:

	- new incident code (0001) indicating a partial approval - modification of the description of the (<tx>)vic_tx_amt and (<tx>)curcy fields Other: - update logo and brands - Accor Services becomes Edenred in the vic_bit_map_application_id table - remove old references to C-ZAM (SPIN and SMASH) - add reference to new products (YOMANI, XENTISSIMO, XENOA) - add curcy field optional in vmc_purse_request, to be used for contactless transactions - field brand_name content limited to 16 bytes - brand_id table exported to another document - new vic_data type (12, floor limit) - update Inter character time-out to 200ms - field hardware_type removed (was previously optional in pdv_command_reply) - incident codes 2500, 4003, 5005 removed (not used anymore) - remove vic_data types 01 (cash back amount), 08 (driver code), 09 (client reference), 10 (client message and 11 (replacement car data)
Version 1.07/13-01	Add new value for KADOZ in vic_bit_map_application_id Add new value for AWL Partners in vic_bit_map_application_id Add new value for RES in vic_bit_map_application_id Add new values for Yourgift, Giftcard and ORES in vic_bit_map_application_id
Version 1.07/13-02	Externalize vic_bit_map_application_id values list to Common Parameters addendum
Version 1.07/13-03	New function_cc value for *cancellation of unspecified service*

Table of contents

1. DOCUMENT SUMMARY 3

1.1. RELEASE MANAGEMENT 4

2. GLOSSARY 9

3. INTRODUCTION 10

3.1. DOCUMENT PURPOSE 10
3.2. RELATED DOCUMENTS 10

4. GLOBAL CONCEPTS 11

4.1. WHAT IS VIC ? 11
4.2. USEFULNESS OF VIC 11

5. VIC APPLICATION PROTOCOL 12

5.1. PROTOCOL CHARACTERISTICS 12
5.2. CONDITION CODES 13
5.3. MESSAGE DESCRIPTION 14
5.3.1. vmc_purse_request 14
5.3.2. pdv_purse_result 15
5.3.3. vmc_debit_request 16
5.3.4. pdv_debit_result 17
5.3.5. vmc_cancel 18
5.3.6. pdv_error 19
5.3.7. vmc_command 19
5.3.8. pdv_command_reply 21
5.3.9. vmc_data 21
5.3.10. pdv_data 23
5.3.11. vmc_state_request 23
5.3.12. pdv_state_info 24
5.4. FIELD FORMAT 25
5.4.1. a field 25
5.4.2. b field 25
5.4.3. l field 25
5.4.4. x field 25
5.4.5. llvar field 25
5.4.6. lllvar field 25
5.4.7. Summary 25
5.5. DICTIONARY 26
5.5.1. acquirer_id 26
5.5.2. additional_amount 26
5.5.1. additional_info_bit_map 26
5.5.2. brand_id 26
5.5.3. brand_name 27
5.5.4. card_id_disp 27
5.5.1. card_token 27
5.5.2. counter_tab_pa 28
5.5.3. curcy 29
5.5.4. cvc2 29
5.5.5. cvm 29
5.5.6. date_and_time 29
5.5.7. display_data 30
5.5.1. display_text_ref 30
5.5.2. ecr_nr 30
5.5.3. function_cc 30
5.5.4. iep_tx_inc 31
5.5.5. iso2_track 32

5.5.6. lg_cust 32
5.5.7. operator_nr 32
5.5.8. pdv_clct_nr 33
5.5.9. pdv_gen_tx_nb 33
5.5.10. pdv_state 33
5.5.11. pdv_state_mask 33
5.5.12. pur_bal 34
5.5.13. sam_gen_tot 34
5.5.14. term_id 34
5.5.15. ticket_data 35
5.5.16. transaction_identifier 35
5.5.17. transaction_protocol 36
5.5.18. tsc_id 36
5.5.19. tx_type[2] 36
5.5.20. vic_bit_map 36
5.5.21. vic_bit_map_application_id 37
5.5.22. vic_card_ind 37
5.5.23. vic_cmd_code 38
5.5.24. vic_config_clct_amt 38
5.5.25. vic_cust_ind 39
5.5.26. vic_data 39
5.5.27. vic_msg_code 40
5.5.28. vic_msg_type 41
5.5.29. vic_proprietary_data 42
5.5.30. vic_pos_entry_mode 42
5.5.31. vic_protoc_id 42
5.5.32. vic_to 43
5.5.33. vic_to_next_request 43
5.5.34. vic_tx_amt 43
5.5.35. vic_tx_id 43
5.5.36. vic_version 45
5.5.37. vmc_type (only relevant if PDV is XENTEO or XENOA) 45
5.6. MESSAGES CONTENTS SUMMARY 46

6. LINE PROTOCOL : KISS 47

6.1. FRAME LAYOUT 47
6.2. CONTROL CHARACTERS 48
6.3. CRC COMPUTATION 51
6.4. TIME OUT CONTROLS 53
6.5. RETRY COUNTER 53
6.6. BYTE STUFFING 54
6.7. TRANSMISSION CONTROL SEQUENCES 54
6.8. ERROR RECOVERY 56
6.9. FLOW CHART 57
6.10. LINE CHARACTERISTICS 60

2. GLOSSARY

BC/MC	Bancontact/MisterCash, commercial name of payment with debit card in Belgium
Closing	(in the context of message vmc_command) The terminal requests an acquirer for the financial information of the current period and the closing of the period
Consultation	(in the context of message vmc_command) The terminal requests an acquirer for the financial information of the current and/or previous period
CHIPKNIP	Commercial name of IEP / payment by purse in the Netherlands
CSM	Chip Security Module
Collection	Transfer of electronic value from a PDV to the host, including the transfer of other electronic data between the terminal and host and vice versa during collection phase.
ECR	Electronic Cash Register
EFT	Electronic Fund Transfer
EMV	Eurocard Mastercard Visa
IEP	Intersector Electronic Purse, an electronic purse which can be used in different sectors thus containing the equivalent of electronic money.
PDV	Purchase Device, Atos Worldline terminal allowing to perform a purchase with a purse.
PINNEN	commercial name of payment with debit card in the Netherlands
PROTON	Commercial name of IEP / payment by purse in Belgium
Purse	Integrated chip card containing an IEP application
Purse balance	The amount of electric money present in a purse
SAM	Secured Application Module, used at PDV level
SIS	Social Identity System
TINA	Temporary Interrupt Application
TPA	Third Party Application (added application)
TSC	Terminal Security Component
TX	Transaction
VIC	Vending-IEP Communication standard
VMC	Vending Machine Controller
XENOA	commercial name of payment terminal
XENTA	commercial name of payment terminal
XENTEO	commercial name of payment terminal
XENTISSIMO	commercial name of payment terminal
YOMANI	commercial name of payment terminal

3. INTRODUCTION

3.1. Document Purpose

The purpose of this document is to provide the VMC (Vending Machine Controller) suppliers and ECR (Electronic Cash Register) suppliers with a detailed specification of the VIC (Vending-IEP-Communication) standard as used in XENTA, XENTEO, XENTISSIMO, YOMANI and XENOA terminals.

The VIC standard was originally intended for Proton transactions between a PDV (Proton Purchase Device) and a VMC. The standard has gradually grown to support more services and devices.

WARNINGS:

- In order to improve our service, the information of this document is subject to change.
- In this document we sometimes use the generic abbreviation IEP (Inter-sector Electronic Purse), the commercial name of the product being **PROTON in Belgium and CHIPKNIP in the Netherlands**.
- In this document we use '*External Device'* to designate the VMC or the ECR (often a PC) and terminal to designate the XENTA, XENTEO, XENTISSIMO, YOMANI and XENOA-terminals manufactured by Atos Worldline.
- The Atos Worldline terminals support part or all of the VIC standard, depending on their specifications.

ADDITIONAL information can be found on the Internet:

- atosworldline.be

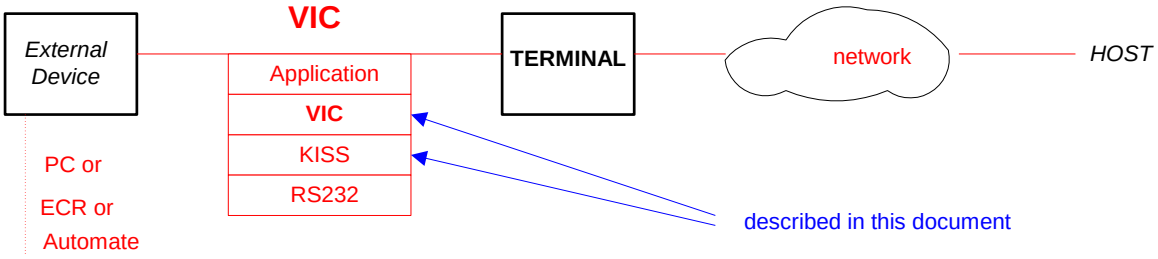
3.2. Related documents

[VIC_vmc_type]	Vmc_type list describes all existing values of this field
[VIC_iep_tx_inc]	Iep_tx_inc list for added applications; describes the specific range reserved for TPA.
[VIC_TINA_EMV]	An Addendum to VIC 1.07.13 for the Emergency fallback (TINA EMV)
[vic_common]	Addendum VIC 1.07 - Common parameters
[text_references]	List of text references

4. GLOBAL CONCEPTS

4.1. What is VIC ?

VIC is an application protocol allowing the communication between a terminal and an External Device. When a terminal is integrated with an External Device, the External Device talks to the terminal via messages. These messages and their use are described in Section 5.



In order to assure reliable and delimited communication between External Device and terminal, messages are exchanged using a telecommunication transportation protocol – KISS – described in Section 6.

4.2. Usefulness of VIC

VIC is used to perform different kinds of transactions;

☑ Payment transactions like
- purse (eg. Proton, Chipknip)
- debit (eg. Bancontact/MisterCash, Maestro, Pinnen)
- credit (eg. Visa, Mastercard,...)
- …

☑ Loyalty transactions
☑ PLC transactions
☑ Health sector transactions like SIS (Social Identity System)
☑ …

5. VIC APPLICATION PROTOCOL

This chapter describes the VIC application protocol, i.e. the protocol of the highest level, between the *External Device* and the terminal.

5.1. Protocol Characteristics

The dialogue between the *External Device* and the terminal uses messages.
The *External Device* is considered as the master and the terminal as the slave: most of the exchanges of messages are done at the initiative of the *External Device*. The contents and use of the different messages are explained in chapter 5.3 Message Description.

The data elements carried by the messages are called fields (eg. field *vic_tx_amt* contains the transaction amount). The contents and use of these fields are explained in chapter 5.4 Field format.
Each field has a unique serial number for all messages. Some fields are not always present (see the condition codes in chapter 5.2 Condition Codes)

An example of a message is a *vmc_debit_request*, which is the most important command of the *External Device* to the terminal*:* it allows performing a transaction. Like any other message, this message carries a message identifier (to be able to distinguish it from other messages). It also carries the requested amount and other information explained further. The *vmc_debit_request* message is transmitted from the *External Device* to the terminal. The communication protocol will indicate to the *External Device* that the terminal has received the message. When the terminal receives the *vmc_debit_request* message, it will

- Identify the message (recognise the message)
- Check the message syntax (check the length, check the message contents format)
- Check the context (e.g. a *vmc_debit_request* is refused if the terminal is collecting)

If these checks do not succeed, the message is considered to be bad and the terminal will transmit a *pdv_error* message to the *External Device*.
If these checks do succeed, the terminal will attempt to perform the debit. The result of this attempt will be communicated to the *External Device* through the *pdv_debit_result* message or *pdv_error*.

The 4 messages

- *vmc_debit_request,*
- *vmc_cancel,*
- *pdv_debit_result,*
- *pdv_error*

are mandatory messages. They are always necessary to be able to use a terminal with an *External Device* (Vending Machine or ECR). They form the **minimum protocol.**

Other messages exist, but are only required for certain features:

- The *vmc_purse_request* and its answer, the *pdv_purse_result* are required if the *External Device* wants information from the card before the transaction is performed and can be used to control the user interface (e.g. selection of products) or to know the services available on the card.
- The *vmc_command* and its answer, the *pdv_command_reply*, allow the *External Device* to control the collection, to ask the current counter values concerning the specified application, to ask financial information and to change the default settings of the terminal.
- The *vmc_state_request* and its answer, the *pdv_state_info* allow the *External Device* to know the state of the terminal, e.g. collecting, purse inserted, merchant screen changed.
- The vmc_data and its answer, the pdv_data allow the External Device to exchange data directly with a TPA installed on the terminal. The External Device should use the field vic_bit_map_application_id to target the right application

Remark:

Every message used in this protocol contains fields which allow evolution:

- an identifier of the protocol version : field *vic_protoc_id*
- an identifier of the protocol subversion : field *vic_version*
- an initial field which indicates which fields will follow : field *vic_bit_map*

5.2. Condition Codes

In the message description in the next paragraphs, several condition codes are used. The table depicted below gives an overview.

Conditions	
m	mandatory
o	optional

5.3. Message Description

This section list the transaction related messages, then the other messages in alphabetical order.

5.3.1. vmc_purse_request

☞ **P**urpose

The *vmc_purse_request* message is used by the *External Device to get card informations in* the following cases:

- Before asking for purse debit, the *External Device* wants to know:
  - Whether a valid card has been read
  - And/or the purse balance
  - And/or the cardholder's language
  - And/or the purse identifier
  - Services supported by the card (vic_bit_map_application_id)
    Examples:
    - A discount is given to certain cardholders on the basis of their card number
    - The card number is recorded before access to a service for which the payment will be done later. This allows for a service such as car parking without ticket (the management is being done by a VMC).

- To know if there is a purse present in the reader, the *External Device* sends a *vmc_purse_request.*

☞ **M**essage layout

Field Nr	Name	Condition
-	vic_protoc_id	m
-	vic_msg_code	m
1	vic_bit_map	m
3	vic_tx_amt	o
16	vic_card_ind	m
20	vic_to	m
23	curcy	o
24	vic_to_next_request	o
171	vic_bit_map_application_id	o
179	vic_version	m
181	additional _info_bit_map	o

Remarks:

vic_tx_amt and *curcy* should be present for contactless transaction. Not used in other case

vic_to represents the time during which the terminal waits for the introduction of the card.

vic_to_next_request represents the time during which the terminals waits for the next_request from the External Device. Default value of this time-out is 30 seconds

vic_purse_info_bit_map represents specific card information requested by the purse. As a response to the request, the *pdv_purse_result* shall contain the fields associated to the bits set in the bitmap. In the case the explicitly requested information cannot be provided (e.g. missing key for the generation of the card_token), an incident code 5030 (Card information not available) shall be reported. Note that in the case of incident code 5030, a *pdv_error* is sent instead of a *pdv_purse_result* (no card recognition is performed when a token is requested).

☞ **C**onditions for sending

This message is optional for the execution of a transaction.
If *a vmc_debit_request* is received in a time slot of the value sent in *vic_to_next_request (default* 30 seconds in case the vic_to_next_request was not present in the *vmc_purse_request*) after a *pdv_purse_result* message, the terminal continues considering the vmc*_purse_request* is part of the transaction session and thus uses the card already inserted/swiped for the transaction; otherwise the two messages are treated as separate transactions.

5.3.2. pdv_purse_result

☞ **P**urpose

The *pdv_purse_result* message is used by the terminal to indicate the presence of a card and the purse balance if applicable.

☞ **M**essage layout

Field		Condition
Nr	Name	
-	vic_protoc_id	m
-	vic_msg_code	m
1	vic_bit_map	m
4	iep_tx_inc	m
5	lg_cust	m
9	pdv_state	m
10	pur_bal	m
11	card_id_disp	m
23	curcy	m
141	vic_pos_entry_mode	o
142	iso2_track	o
171	vic_bit_map_application_id	m
179	vic_version	m
180	card_token	o

Remarks:

card_id_disp can be equal to zero for EMV cards.

lg_cust can be equal to zero for EMV cards.

If a card without a proton purse is read the purse balance (*pur_bal*) is equal to 00 00 00.

curcy is the currency of the card if there is one. For magnetic card this is the currency of the terminal.

vic_pos_entry_mode is a conditional field. This field will be present if the field vic_tx_amt is present in the request.

vic_bit_map_application_id is a bitmap. Several bits can be set to indicate the card is a multiservice card ie : proton purchase(value : 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 08) + BC/MC (value: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 01) = 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 09.

card_token is a conditional field. It will be present if requested by the vic_purse_info_bit_map of the request. If this field is present, no bits are set in the *vic_bit_map_application_id* (no card recognition is performed) and lg_cust, curcy and card_id_disp have a default value.

☞ **C**onditions for sending

It is sent by the terminal as reply to a *vmc_purse_request* message sent by the *External Device.*

5.3.3. vmc_debit_request

Purpose

The *vmc_debit_request* message is used by the External Device to ask the terminal for the debit of a card or an account.
The External Device can have a certain control on the means of payment used for the transaction, but we do not recommend it: this can be done by adding the optional field vic_bit_map_application_id with one or more bits set according to the means of payment the external device wants to accept.
If a card with multiple means of payment is used for the transaction, the terminal will propose a list.

Message layout

Field		Condition
Nr	Name	
-	vic_protoc_id	m
-	vic_msg_code	m
1	vic_bit_map	m
3	vic_tx_amt	m
12	tx_type	m
16	vic_card_ind	m
20	vic_to	m
21	vic_tx_id	m
23	curcy	m
25	vic_cust_ind	m
35	vic_proprietary_data	o
36	vic_data	o
142	iso2_track	o
143	operator_nr	m
144	function_cc	o
146	ecr_nr	o
171	vic_bit_map_application_id	o
177	cvc2	o
179	vic_version	m
181	additional_info_bit_map	o

Remarks:

If *function_cc* is not present for an operation with a credit card the default function is ‘sale’.

vic_data is used for adding references that need to be sent to the host and/or need to be printed on the transaction ticket

vic_cust_ind indicates whether the client has to approve the transaction – by using the <OK> key – before it happens (for purse payments only and only if permitted by the acquirer).

vic_to represents the time during which the terminal waits for introduction of the card.

vic_purse_info_bit_map represents specific card information requested in the response: the *pdv_debit_result* shall contain the fields associated to the bits set in the bitmap. In the case the explicitly requested information cannot be provided (e.g. missing key for the generation of the token), an incident code 5030 (Card information not available) shall be reported. Note that in the case of incident code 5030, a *pdv_error* is sent instead of a *pdv_purse_result* (no card recognition is performed when a token is requested).

☞ Conditions for sending

This message is required for the execution of a transaction with a payment card.
In the exceptional case of no response received from the terminal after acknowledgement of the *vmc_debit_request*, the *vmc_debit_request* shall not be repeated but the *External Device* has to send a *vmc_cancel* message. Typical time-out value are 90 and 180 seconds (depends on the communication means used between the terminal and the host).

5.3.4. pdv_debit_result

☞ Purpose

This message gives the result of the debit request or can also be the answer to a *vmc_cancel* message.

☞ Message layout

Field		Condition
Nr	Name	
-	vic_protoc_id	m
-	vic_msg_code	m
1	vic_bit_map	m
2	term_id	m
3	vic_tx_amt	m
4	iep_tx_inc	m
5	lg_cust	m
7	pdv_clct_nr	o
8	pdv_gen_tx_nb	o
9	pdv_state	m
11	card_id_disp	m
13	sam_gen_tot	o
21	vic_tx_id	m
22	vic_msg_type	m
23	curcy	m
35	vic_proprietary_data	o
36	vic_data	o
125	<tx>vic_tx_amt	m
126	<tx>curcy	m
145	ticket_data	o
159	display_data	o
167	acquirer_id	o
169	additional_amount	o
170	transaction_protocol	o
171	vic_bit_map_application_id	m
172	transaction_identifier	o
173	date_and_time	o
174	brand_id	o
176	cvm	o
178	brand_name	o
179	vic_version	m
180	card_token	o
182	display_text_ref	o
183	tsc_id	o

Remarks:

vic_data is present when the Acquirer sends a SDD and/or a reference and/or an authorization code.

lg_cust is the language of the card. Is present if requested in the vic_purse_info_bit_map field of the request.

The additional_amount is in the currency of curcy field. To have the total of the transaction, look at the field *<tx>vic_tx_amt*.

The *card_id_disp* is the card_id read on the card on the terminal side or sent by the host. Is present if requested in the vic_purse_info_bit_map field of the request.

The fields *pdv_state* and *sam_gen_tot* are present but only relevant for purse payments.

The *brand_name* provides the brand label name to the External Device, this label has to be used when printing a transaction ticket.

card_token is a conditional field. It will be present if requested by the vic_purse_info_bit_map of the request. If this field is present, no bits are set in the *vic_bit_map_application_id* (no card recognition is performed).

tsc_id will be present, if applicable, for an Emergency fallback transaction, in the case the acquirer imposed a TSC security.

☞ **Conditions for sending**

The *pdv_debit_result* is sent to answer a *vmc_debit_request* or a *vmc_cancel* message.

5.3.5. vmc_cancel

☞ **Purpose**

This message cancels certain requests, collects information at the startup of the external device or on the last transaction if there is any doubt on its outcome.

☞ **Message layout**

Field		Condition
Nr	Name	
-	vic_protoc_id	m
-	vic_msg_code	m
1	vic_bit_map	m
31	vmc_type	m
179	vic_version	m

☞ **Conditions for sending**

The *External Device* may send a *vmc_cancel* in the following cases:

- When the *External Device* wants to get information about the last debit request command.
- All *External Devices* should send a *vmc_cancel* to the terminal after a power up
- After a *pdv_purse_result* to avoid using the card data for the next transaction
- [Netherlands only] to stop an ongoing transaction (possible only as long as the authorisation message is not yet sent to the acquirer host and if the payment application can support this functionality).
- [XENOA only] to stop an ongoing transaction at some steps:
  - directly after a card is read and as long as a *vmc_purse_request*/*vmc_debit_request* is not sent
  - directly after a *vmc_purse_request*/*vmc_debit_request* and as long as the card is not read

- after a *vmc_purse_request/vmc_debit_*and the request from the PDV for a card reading retry (e.g. by sending a *pdv_state_info*
- after a *pdv_purse_result* and before a *vmc_debit_request*

5.3.6. pdv_error

☞ **P**urpose

This message indicates an error.

☞ **M**essage layout

Field		Condition
Nr	Name	
-	vic_protoc_id	m
-	vic_msg_code	m
1	vic_bit_map	m
4	iep_tx_inc	m
9	pdv_state	m
22	vic_msg_type	m
179	vic_version	o

☞ **C**onditions for sending

This message is sent in reply to any *External Device* message when this message cannot be accepted by the terminal, e.g. illegal message contents, illegal message length or terminal is not in correct state for this message (e.g. *vmc_debit_request* while terminal not operational).
This message is sent when an error (e.g. card removed or <STOP> pushed) occurs during the card recognition.
This message is sent when an error occurs during a transaction starting with a vmc_data.

Depending on terminal setup (VIC or NewVIC), the terminal may also send a pdv_error if it has been rebooted. In this case:

- The *pdv_error* will be repeated until it is acknowledged by the external device
- The field *vic_msg_type* will indicate that the *pdv_error* is sent due to a terminal reboot
- The vic_version field won't be sent, as the terminal is not yet configure to use any specific version

5.3.7. vmc_command

☞ **P**urpose

This message is used by the *External Device* to request the terminal to execute an action.
The required action can be a proton collection, an exchange of functional parameters between the *External Device* and the terminal, a demand to the terminal to send the current counter values concerning the specified couple application/service.

☞ **M**essage layout

Field		Condition
Nr	Name	
-	vic_protoc_id	m
-	vic_msg_code	m
1	vic_bit_map	m
17	vic_cmd_code	m
18	vic_config_clct_amt	o
19	pdv_state_mask	o

23	curcy	o
167	acquirer_id	o
171	vic_bit_map_application_id	o
179	vic_version	m

Remark:

The command can be directed to an application (use field vic_bit_map_application_id) or to an acquirer (use field acquirer_id).

Conditions for sending

This message is optional.

The *External Device* sends a *vmc_command* message when it wants the terminal to execute an action. Four different types of 'actions' can be requested:

- action : configuration
  - The external device wants to modify a functional parameter of the terminal, e.g. decide the way in which the collection has to be done and/or under which conditions the *External Device* wants to receive the *pdv_state_info* messages. Any configuration change requested with this command will be reset if the terminal has rebooted. Especially, in order to configure the Proton application, the fields vic_config_clct_amt, curcy and vic_bit_map_application_id (with value 0x08) become mandatory.

- action : information
  - The external device wants to activate, deactivate a service on the terminal, or wants to check if a service is activated or not. The field vic_bit_map_application_id identifies the service.

- action : consultation/closing
  - The external device wants to request the counters for the different means of payment and/or close the financial period

- action : Get/Print ticket
  - The external device wants to get the last merchant or client ticket in order to print it.
  - The external device wants to print a mandatory pending ticket on the terminal. For this action the terminal has to have a printer and this printer has to be activated.

Depending on the action type requested, the following rules concerning presence of fields in the message apply :

		Configuration	Information	Consultation/ Closing	Get/Print Ticket
-	vic_protocol_id	m	m	m	m
-	vic_msg_code	m	m	m	m
1	vic_bit_map	m	m	m	m
17	vic_cmd_code	m	m	m	m
18	vic_config_clct_amt	o			
19	pdv_state_mask	m			
23	curcy	o			
167	acquirer_id			m	
171	vic_bit_map_application_id	o	m		
179	vic_version	m	m	m	m

5.3.8. pdv_command_reply

☞ **P**urpose

This message is the response of the terminal to the *vmc_command* message sent by the *External Device*.

☞ **M**essage layout

Field		Condition
Nr	Name	
-	vic_protoc_id	m
-	vic_msg_code	m
1	vic_bit_map	m
4	iep_tx_inc	m
145	ticket_data	o
157	counter_tab_pa	o
167	acquirer_id	o
171	vic_bit_map_application_id	o
179	vic_version	m

☞ **C**onditions for sending

This message is always sent in reply to a *vmc_command* message.
Depending on the action type requested, the following rules concerning presence of fields in the message apply:

		Configuration	Information	Consultation/ Closing	Get/Print Ticket
-	vic_protocol_id	m	m	m	m
-	vic_msg_code	m	m	m	m
1	vic_bit_map	m	m	m	m
4	iep_tx_inc	m	m	m	m
145	ticket_data				o
157	counter_tab_pa			m	
167	acquirer_id			m	
171	vic_bit_map_application_id	o	m		
179	vic_version	m	m	m	m

- action : configuration
  - the vic_bit_map_application_id must be sent back in the response to a configuration *vmc_command* specifying this field; its value must be echoed.

- action : Get/Print ticket
  - if the request is for getting the ticket, the ticket_data field must be present in the response.

5.3.9. vmc_data

☞ **P**urpose

This message is exchanged between External Device and Terminal when data must be transferred from the External Device to a TPA in the Terminal.

Should be used for routing to Added applications

Message layout

Field Nr	Name	Condition
-	vic_protoc_id	m
-	vic_msg_code	m
1	vic_bit_map	m
36	vic_data	m
171	vic_bit_map_application_id	m
179	vic_version	m

Remark:

The field vic_bit_map_application_id identifies the TPA, the field vic_data contains the data to be transferred.

5.3.10. pdv_data

☞ **P**urpose

This message is exchanged between the Terminal and the External Device when data must be transferred from a TPA in the Terminal to an External Device.

☞ **M**essage layout

Field		Condition
Nr	Name	
-	vic_protoc_id	m
-	vic_msg_code	m
1	vic_bit_map	m
36	vic_data	m
171	vic_bit_map_application_id	m
179	vic_version	m

Remark:

The field vic_bit_map_application_id identifies the originating TPA, the field vic_data contains the data to be transferred.

5.3.11. vmc_state_request

☞ **P**urpose

This message is used when the *External Device* wants to request the terminal state.

☞ **M**essage layout

Field		Condition
Nr	Name	
-	vic_protoc_id	m
-	vic_msg_code	m
1	vic_bit_map	m
179	vic_version	m

☞ **C**onditions for sending

This message is optional.

The *External Device* can send a *vmc_state_request* message when it wants to know the terminal state.

5.3.12. pdv_state_info

☞ **P**urpose

This message gives information about the terminal state.

☞ **M**essage layout

Field		Condition
Nr	Name	
-	vic_protoc_id	m
-	vic_msg_code	m
1	vic_bit_map	m
4	iep_tx_inc	m
9	pdv_state	m
22	vic_msg_type	m
159	display_data	o
173	date_and_time	o
179	vic_version	m
182	display_text_ref	o

☞ **C**onditions for sending

This message is sent in the following cases :

- reply to the *vmc_state_request* message
- unsollicited message sent when state of the terminal changed (only if External Device requested this by a vmc_command message for configuration)
- unsollicited message sent when a new merchant text has to be displayed on the External Device (only if configured in Terminal's technical menu)

5.4. Field format

This section lists the data elements format conventions.

5.4.1. a field

ASCII fields need 7 bits per element, they are converted to one byte per character, right justified, and the left most bit is always 0. (0x00->0x7F)

5.4.2. b field

Boolean fields need one bit per element, they are converted to one byte per 8 bits, left justified (bit one at most significant bit position), padded with 0.
A b+ field indicates an extensible field, depending on the value of the first bit (usually used for bitmaps).

5.4.3. I field

Integer fields need 4 bits per element; they are converted to one byte per 2 digits, right justified, padded with 0.(Binary coded decimal BCD)

5.4.4. x field

Hexadecimal fields need 4 bits per element, they are converted to one byte per 2 digits, right justified, padded with 0.

5.4.5. llvar field

In this format, the first byte of the field contains the field length in bytes (ll is of type 2i). The length indicated in ll does not include the length of ll itself.
The total length of a field of format llvar is thus the length of the data in the field + 1 byte.
Example : 03 11 22 33

5.4.6. lllvar field

In this format, the first two bytes of the field contain the field length in bytes (llll is of type 4i). The length indicated in llll does not include the length of llll itself.
The total length of a field of format lllvar is thus the length of the data in the field + 2 bytes.
Example : 0009 11 22 33 44 55 66 77 88 99

5.4.7. Summary

Format	Values	Justification	Padding	Remark
a	7 bits ASCII	right	-	
b	false, true	left	0	false = 0, true = 1
I	0...9	right	0	
x	0...9,A...F	right	0	
llvar	0…9 (ll)	-	-	ll contains length of data
llllvar	0...9 (llll)	-	-	llll contains length of data

5.5. Dictionary

This section lists the data elements in alphabetical order.

5.5.1. acquirer_id

Name : acquirer id
Format : 8i

Definition: This field identifies an acquirer. The different values are defined in [vic_common].

5.5.2. additional_amount

Name : additional amount (Tip)
Format : 6x

Definition : This field contains the amount of the tip granted by the cardholder (present only if a tip was actually given), in the currency of the field <tx>curcy.

5.5.1. additional_info_bit_map

Name : Additional Info bitmap
Format : 32b
Values : each bit corresponds to a specific information in the pdv_purse_result or the pdv_debit_result

bit 0	card_token (180)
other bits	reserved for future use

Definition : The Additional Info Bit Map should be used in a *vmc_purse_request* or *vmc_debit_request*, to specify some specific information expected in the pdv_purse_result/pdv_debit_result sent in response.

5.5.2. brand_id

Name : brand id
Format : 4x

Definition: This field identifies a service. The different possible values are defined in [vic_common].

5.5.3. brand_name

Name : brand name
Format : llvar
Value : any ASCII characters

Definition: The field brand_name contains the brand label name of the card that was used for the transaction, the contents of this field has to be used to format a transaction ticket; maximum 16 characters.

5.5.4. card_id_disp

Name : Card Identification
Format : 24a
Values :
Definition : This field contains an identification of the card inserted in the terminal *in a ready to print format*.

5.5.1. card_token

Name : card_token
Format : lllvar

Definition : This field contains a token generated by the terminal on the basis of information from the card.

5.5.2. counter_tab_pa

Name : financial counter table
Format : llllvar rt
Sub-structure definition

Field	Format	level2_bitmap bit nr
data length (in bytes)	4i	
fld_hdr		1
nb_record	2i	
file_updt_code	1i	
level2_bitmap	16b+	
counter_id	4i	2
counter_curcy	4i	3
counter_name	16a	4
tot_tx	4i	5
tot_amount	8x	6
terminal_period	4i	7
brand_id	4x	9

where:

• **fld_hdr:**	field header composed with
• **nb_record:**	number of record(s) included in this table
• **file_update_code:**	indicates the operation that must be performed on the content of the table
• **level2_bitmap:**	bitmap indicating the presence of the level2 fields
• **counter_id:**	identifies the current counter
• **counter_curcy:**	indicates the currency used for the current counter
• **counter_name**	indicates the name of the current counter
• **tot_tx**	indicates the total number of valid transactions in the current accounting period for the current counter
• **tot_amount:**	indicates the total amount of transactions in the current accounting period for the current counter
• **terminal_period**	indicates the number of the current accounting period
• **brand_id**	identifies the brand associated to the current counter

Definition : This table gives information about one or more counter(s) related to a couple application/service.

5.5.3. curcy[1]

Name : Currency Code
Format : 4i
Values : Euro (EURcent) 0978

Definition : curcy is the currency of the requested transaction amount.
<tx>curcy is the currency of the effectively approved transaction amount.

5.5.4. cvc2

Name : Card validation code 2
Format : 4i
Values : [0000..9999]

Definition : The cvc2 may be required for certain type of EMV credit card transactions depending on the acquirer.

5.5.5. cvm

Name : cardholder verification method
Format : 4x
Values : xx AB

A

With cardholder signature required	**1**
Without cardholder signature required	**2**
Default	**0**

B

With pin	**1**
Without pin	**2**
Default	**0**

The default value for the most significant byte is 0x00. Other values are not yet used.
Definition: This field informs the external device about the way the transaction was validated.

5.5.6. date_and_time

Name : Date and time
Format: 14i
Values: The format of this field shall be “YYYYMMDDhhmmss” where “YYYY” are the four digits of the year, “MM” are the two digits of the month, “DD” are the two digits of the day, “hh” are the two digits of the hour, “mm” are the two digits of the minutes, and “ss” are the two digits of the seconds.

Definition: The value refers to the time of authorisation by the acquirer.
For TINA, Reference to [BKS D110-TN1]: date and time
For Power Management features: refers to next date and time the Purchase Device must be waked up

[1] Or <tx>curcy

5.5.7. display_data

Name : display_data
Format : lllvar

Definition : This field contains data that needs to be displayed on the merchant screen of the External Device. This field contains only printable characters represented hexadecimally (eg. 0x41 = A) as well as the character “line feed” 0x0A
Note : the EUR-sign € is sent as 0xA4.

5.5.1. display_text_ref

Name : display_text_ref
Format : 4x

Definition : This field contains a reference representing a specific message to be displayed on the VMC, as defined in [text_references].

5.5.2. ecr_nr

Name : ECR nr
Format : 2i

Definition : number of the Electronic Cash Register.
Some multi-terminal configurations use this value to identify a terminal in a more user friendly way.

5.5.3. function_cc

Name : Function code
Format : 2a
Values :

“ ” (0x20, 0x20)	sale
“00” (0x30, 0x30)	sale cancellation
“01” (0x30, 0x31)	*reserved*
“02” (0x30, 0x32)	reservation
“03” (0x30, 0x33)	sale after reservation
“04” (0x30, 0x34)	reservation cancellation
“05” (0x30, 0x35)	sale after reservation cancellation
“06” (0x30, 0x36)	voice referral
“07” (0x30, 0x37)	*reserved*
“08” (0x30, 0x38)	*reserved*
“09” (0x30, 0x39)	card validity check
“10” (0x31, 0x30)	*reserved*
“11” (0x31, 0x31)	refund
“12” (0x31, 0x32)	cash advance
“13” (0x31, 0x33)	*reserved*
“14” (0x31, 0x34)	cancellation of unspecified service

Definition : This field identifies the type of transaction that will be sent to the acquirer. The values are under the responsibility of the acquirers and are not interpreted by the terminal Associated to the selected function code, a reference (like an authorization code) can be needed depending on the acquirers: This reference needs to be sent in the field vic_data.

5.5.4. iep_tx_inc

Name : Transaction Incident
Format : 4i
Values :

Result OK:	
No incident encountered	0000
Requested transaction amount only partially approved	0001
Request not taken into account terminal busy:	
Bad protocol state(terminal busy with the previous action)	4009
Result Not Ok:	
Technical Problem	1501
Entered amount invalid	1503
iep_host_id not valid	1504
pur_id_ac_pub error	1505
purse in red list	1506
purse is locked for credit	1508
purse is locked for debit	1509
Purse expired	1512
ICC state error	1513
recovery error	1516
purse key identifier error	1517
purse balance is too large	1518
pur_id_ext_ac_pub error	1520
No paper in the printer	2001
Mandatory ticket pending	2002
No purse in reader and time out expired	4001
No validation by the customer and time out expired	4002
Request aborted by the VMC	4004
Insufficient purse balance	4005
Bad value in a field	4006
Invalid message length	4008
Application not supported	4010
Time-out on fallback card reading	4011
Technical problem	5001
Rejection by the host	5002
Double operation (consecutive transactions with same amount and same card)	5003
Technical problem at host level	5004
Stop customer	5006
Invalid currency	5008
Rejection by the terminal	5009
Communication problem (wrong phone nr,...)	5010
Floor limit exceeded	5012
Transaction refused by the terminal in EMV mode	5013
Transaction refused by the card	5014
The maximum transaction number allowed per month is reached	5016
The maximal number of uncollected journals in terminal reached	5017
Service (already) activated	5018
Service (already) deactivated	5019
Acquirer does not support service activation	5020
Maximum transaction records is reached	5021
Maximum number of TINA activation reached	5022
Amount higher than authorized amount	5023
Problem linked to card	5024
Card Blocked	5025
Card not authorized	5026

Incorrect PIN	5027
Product not allowed	5028
Product not available	5029
Card information not available	5030
Incident codes for Added Application 7000 →7999	
see document [VIC_iep_tx_inc]	

Definition : The Transaction Incident indicates if the operation succeeded or not. If the operation didn't succeed, it indicates the kind of problem encountered.

5.5.5. iso2_track

Name : Iso2 track field
Format : 38x

Definition : 1. This field must be used in case of manual entry of the card number (eg. mail order sale) : the first byte should be 0x5D ("]"),the card number should be terminated by D hex and followed by the date (mmyy). The rest has to be padded with "NULL" (0x00)

2. This field must be used in case of bar code entry : the first byte should be 0x5B ("[") and the bar code number. The rest has to padded with "NULL" (0x00).

5.5.6. lg_cust

Name : Customer (Card Holder) Language
Format : 1i
Values :

Unknown	0
Dutch	1
French	2
German	3
English	4

Definition : This field indicates the language of the card holder.

5.5.7. operator_nr

Name : Operator identifier
Format : 4i
Values : [0001..9999]

Definition : This number identifies the operator.
Some multi-terminal configurations use this field to manage counters per operator

5.5.8. pdv_clct_nr

Name : PDV Collection Number
Format : 3i
Excepted 4i for
- TINA and Emergency fallback transactions
- Transactions managed via the EFT Server (IFSF)

Values :

not defined	000 or 0000
otherwise	[001..999] or [0001..9999]

Definition : Serial number of the accounting period, returns to 1 after 999 (or 1 to 9999), not necessarily present/relevant for all types of transactions

5.5.9. pdv_gen_tx_nb

Name : General Transaction Number
Format : 8x
Values : [00000001.. FFFFFFFF]

Definition : Counter of transactions in the current accounting period, not necessarily present/relevant for all types of transactions

5.5.10. pdv_state

Name : Purchase Device State
Format : 32b
Values : When the condition is true, the corresponding bit is set to 1

bit 2	Card or object inserted
bit 3	Wake up date/time changed
bit 12	Purse collection started
bit 15 & bit 16	Purse collection error
other bits	not relevant

Definition : The Purchase Device State indicates the function state of the specified application in the terminal.

5.5.11. pdv_state_mask

Name : Purchase Device State Mask
Format : 32b
Values : When the condition is true, the corresponding bit is set to 1

bit 2	Card or object inserted
bit 3	Purchase Device wake up activated
bit 4	VMC display update request activated
bit 12	Purse collection state
other bits	not used

Definition : Every bit set in this field triggers an unsollicited pdv_state_info message from the terminal to the external device when the corresponding bit has changed.

Special case: Setting bit 12 triggers unsollicited messages for bit 12, 15 and 16.

5.5.12. pur_bal

Name : Purse Balance
Format : 6x
Values : [000000 .. FFFFFF]

Definition : The Purse Balance defines the amount of electronic money in the Purse of the inserted card. The purse balance is expressed in the currency of the field curcy.
This field is left zero if the card doesn't contain the Purse service

5.5.13. sam_gen_tot

Name : SAM General Total
Format : 8x
Values :

Definition : The SAM General Total defines the total amount of Electronic Money collected for all the Purchase Transactions in the transaction currency since the very beginning of the SAM life.

5.5.14. term_id

Name : Terminal Identification
Format : 8a

Definition : A Terminal Identification is assigned to every terminal.
It is used to identify the terminal administratively.

5.5.15. ticket_data

Name : Ticket data
Format : llllvar a

Definition : This field contains (in a "ready for print" format) the data of the transaction ticket.
The maximum width of printable data will not exceed 38 chars.
The maximum length of this field is 999 bytes.

The special characters the ticket_data may contain are:

0x04 : use double height characters from here till the end of the line, cancelled by 0x14
0x0E : use double width characters from here till the end of the line, cancelled by 0x14
0x14 : cancel expanded mode

0x0A : line feed
0x0D : carriage return
0x0B : vertical tabulation: advance n lines
0x0C : form feed

USED ??0x1B0x4C : set left margin to position 0=<n=<38
<ESC> 'N' : (1Bhex 4Ehex) : set bottom margin to n lines
<ESC> 'O' : (1Bhex 4Fhex) : cancel bottom margin set by ESC 'N'
<ESC> 'P' : (1Bhex 50hex) : set page length to n lines

Any other non-printable character should be ignored

5.5.16. transaction_identifier

Name : transaction_identifier
Format : 8i
Values : [00000001.. 99999999]

Definition : The transaction identifier field contains a unique reference number that is provided by the Acquirer.

5.5.17. transaction_protocol

Name: transaction_protocol
Format: 4x
Values: **xx AB**

A

Chip	1
Magnetic	2
Magnetic in fall back mode	3
Default	0

B

Off line	1
On line	2
Off line after communication failure	3
Emergency fallback	4
default	0

The default value for the most significant byte is 0x00. Other values are not yet used.

Definition: The field informs external device about the way the transaction was authorised.

5.5.18. tsc_id

Name: Security scheme TSC Identifier
Format: 12x
Values: [00 00 00 00 00 01..FF FF FF FF FF FF]$_H$

Definition: Identifies the TSC with respect to the configured security scheme used for the transaction.

5.5.19. tx_type[2]

Name : Purse Transaction Type
Format : 2x
Values:

Normal transaction	04_H
Force on line transaction (credit transaction)	08_H

Definition : This field is used to define transaction type.

5.5.20. vic_bit_map

Name : VIC Bit Map
Format : 64b+ (multiples of 64 bits)

Definition : The bit corresponding to the field number is set for every field present in a message. The first bit of each 64 bits group is an exception: it is set if another 64 bits bitmap is following this one (allowing extensions of the vic_bit_map field size).

5.5.21. vic_bit_map_application_id

Name : VIC bit map application identification
Format : 128 bits
Values : The different possible values are defined in [vic_common].

Definition : This field is a bit map. The position of the bit set, corresponds to an identification value. This value identifies the application(s) present on a card with multiple applications. The providers assign a unique identification to every application or brand.
Note that the list is subject to change at any time; the ECR should then be able to accept new and not yet known vic_bit_map_application_id's.

For Example: In a *pdv_purse_result*, if the card inserted is a hybrid card with BC/MC and proton purchase services and the terminal has these services activated, the *vic_bit_map_application_id* will be 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 09.

vic_bit_map_application_id present in message coming from the *External device*	Action on terminal
no	routed to the default application. In case multiple card services are supported by the card, the choice is proposed to the cardholder on the terminal.
yes	Check the compatibility between the bit set by the ECR, the services on the card and the application recognition. If it succeeds the terminal proposes services or payments choice.

As opposed to other application ID, the Emergency fallback (TINA EMV) ID must be used in *pdv_debit_result* in association with another Common Application ID: in this case, the vic_bit_map_application_id will contain a double information (two bits set). It may also be used in *vmc_command* for service activation/deactivation, as for TINA. It shall not be used in other messages.

5.5.22. vic_card_ind

Name : VIC Card Indicator
Format : 1i
Values :

Card_withdrawal_not_requested	0
Card_withdrawal_requested	1
Card_not_needed	5
Display_brand_choice and card_withdrawal_requested [addendum vic pass]	6

Definition : The VIC Card Indicator indicates whether the customer will be asked to retrieve his card or not after the terminal has performed the action requested by the *External Device*. Normally, the value is 1 for a *vmc_debit_request* and 0 for a *vmc_purse_request*. *Card_not_needed* can be used e.g. when asking totals for Credit Card to avoid the reading of a card on the terminal side.

The vic_card_ind is not taken into account in all messages.

5.5.23. vic_cmd_code

Name : VIC Command Code
Format : 4x
Values :

Configuration	$C000_H$
Consultation for terminal only	$B000_H$
Consultation for complete site	$B001_H$
Closing for terminal only	$D000_H$
Closing for complete site	$D001_H$
Service activation request	$E000_H$
Service deactivation request	$E001_H$
Service consultation request	$E002_H$
Get last merchant ticket	$F001_H$
Get last client ticket	$F002_H$
Print pending merchant ticket	$F003_H$
Print pending client ticket	$F004_H$

Definition : The VIC Command Code indicates the action the *External Device* requires the terminal to do.

5.5.24. vic_config_clct_amt

Name : VIC Configuration Collection Amount
Format : 8x
Values :

Collection initiated by the terminal	00000000_H
Automatic collection by the terminal next night when this amount is reached	$[00000001..000FFFFF]_H$
Not supported	$[00100001..FFFFFFFE]_H$
Collection initiated by the *External Device* [2]	$FFFFFFFF_H$

Example : 500 € is equivalent to $0000C350_H$

Definition : The VIC Configuration Collection Amount indicates whether the collection is initiated by the *External Device* or by the terminal or automatically when a specified amount is reached.

[3] In this case the *terminal* will not start a collection on its own initiative. A collection can however be started via VIC or via the keyboard (for operator).

5.5.25. vic_cust_ind

Name : VIC Customer OK Indicator
Format : 1i
Values :

Customer_OK_needed	1
Customer_OK_unneeded	0

Definition : The Customer OK Indicator indicates whether or not the customer has to approve the transaction–- by using the <OK> key–- before it happens.
Acquirer has to approve the setting of this parameter.
The default value is 1.

Only used in case of Purse transaction.

5.5.26. vic_data

Name : VIC data
Format : lllllvar
Max length: 999

Sub-structure definition in case of transfer to or from acquirer host (discretionary data)

Field	**Format**	**Value**
data length (in bytes)	4i	0001..0999
set_of_data (SD)	llvar	

The SD field is composed of:

Subfield	**Format**	**Value**
data length (in bytes, this byte not include)	2i	01..99
Type code	2i	See description below
Data	See description below	

Type code value[3] :

02: presence of **product code shop**

Only used in case of Company cards.

Shop_code	2i	
Shop_amount	6x	currency = curcy

03: presence of **petroleum data**

Only used in case of Company cards.		
Volume (expressed in centilitres)	6i	
Unit_price (xxx.xxx EUR/litre or xxxx.xx BEF/litre)	6x	currency = curcy
Product_code	2i	
Product_name	16a	

Maximum 16 different shop and petrol data are allowed

[3] the types in grey are currently not used

04: presence of a set of **merchant reference**

Merchant references are reference that the merchant can use to have a personal mechanism to identify the transaction.

length max 98
Reference1 Reference2 Reference3 … Var a

05: presence of discretionary data

length max 80
discretionary_data Var a

06: presence of an **authorization code**

The authorization code is a reference that the host uses to identify an operation. If the merchant wants make reference of a previous operation (e.g. a sale) during the current operation (e.g. cancel of the sale), it uses the authorization code of the previous operation

length max 6
Authorization Code Var a

07: presence of a **mileage**

The mileage is the mileage information entered by the customer during a transaction with a company card.

length max 8
mileage Var i

12: presence of a **floor limit**

The Floor limit is a maximum limit that can be used to perform a transaction (regardless of which mean of payment or which type of transaction).
No additional currency indication (<tx> curcy field is already present in the message)
For a limit expressed in EURO the amount will be in eurocents.

Floor limit 12 i

A combination of codes in the same instance of *vic_data* is allowed.
e.g : **00543304***reference1;reference2;reference3***1105***abcde12345***0706***auth12*

5.5.27. vic_msg_code

Name : VIC Message Code
Format : 2a
Values :

vmc_purse_request	VP
pdv_purse_result	PP
vmc_debit_request	VD
pdv_debit_result	PD
vmc_cancel	VC
pdv_error	PE
vmc_command	VM
pdv_command_reply	PM

vmc_data	VO
pdv_data	PO
vmc_state_request	VS
pdv_state_info	PS

Definition: The VIC Message Code identifies the message exchanged between the *External Device* and the terminal.

5.5.28. vic_msg_type

Name : VIC Message Type
Format : 8b
Values :

Default	0000 0000
Duplicate message	1xxx xxxx
Unsolicited	x1xx xxxx
Startup	xx1x xxxx
Display message	xxx1 xxxx
	x = not relevant

Definition : The VIC Message Type provides additional info about the reason why the message was sent :

- Unsollicited is used to indicate that the terminal sent the message without request from the External Device.
- 'Startup' is used to indicate the *pdv_error* is sent because the terminal is starting up.
- 'Duplicate message' is used to indicate the *pdv_debit_result* is a repetition[4] of a previously transmitted identical message (possibly not received by the *External Device*).
- 'Display message' is used to indicate that the terminal sent this message because a new message (in display_data or display_text_ref) has to be shown to the merchant (unsollicited)

5.5.29. vic_proprietary_data

Name : VIC proprietary data
Format : llllvar
Max length: 999
Values :

Definition : The VIC Proprietary Data contains specific data for proprietary transaction.

In the particular case where this field is transmitted in a *pdv_debit_result* message where the vic_bit_map_application_id field indicates the transaction has been performed via the Emergency fallback (TINA EMV), this field has to be decoded as specified in [VIC_TINA_EMV].

5.5.30. vic_pos_entry_mode

Name : VIC Point-of-service entry mode
Format : 2i
Values :

Code	Meaning
00	Unspecified (undefined or unknown)
01	Manual entry
02	Magstripe entry
05	Icc (Chip with contact)
07	Contactless Chip
91	Contactless Mag Stripe

Definition : The VIC Point-of-service entry mode inform about the way that the card information data was entered or obtained.

5.5.31. vic_protoc_id

Name : VIC Protocol Identification
Format : 4x
Values : VIC protocol version 01.07 0107_H

Definition : The VIC Protocol Identification identifies the protocol version for compatible upgrade management.
The *External Device* software shall take this into account and check on the value of *vic_protoc_id* sent by the terminal.

5.5.32. vic_to

Name : VIC Time Out
Format : 4x
Values :

immediate response	0000H
wait delay **(in seconds)**	$[0014..002D]_H$

Definition : The VIC Time Out indicates the maximum time available to the terminal to execute the reading of the card **in seconds**.

5.5.33. vic_to_next_request

Name : VIC Time Out Next Request
Format : 4x
Values :

wait delay **(in seconds)**	$[0005..FFFF]_H$

Definition : The VIC Time Out Next Request indicates the maximum time between a pdv_purse_result and the next command usually a vmc_debit_request **in seconds**
If this field is not present, the default time out of 30 seconds is used

5.5.34. vic_tx_amt[4]

Name : VIC Transaction Amount
Format : 6x
Values :

Definition : The vic_tx_amt defines the amount requested in the current transaction, in the currency defined by the field curcy.
The field <tx>vic_tx_amt contains the amount effectively approved during the transaction, in the currency defined by the field <tx>curcy.

If a tip is granted by the customer, the field <tx>vic_tx_amt will contain the amount with tip, the field vic_tx_amt will contain the original amount . The field additional_amount contains the tip amount.

If the transaction is partially approved, the field <tx>vic_tx_amt will contain the effectively approved amount and the field vic_tx_amt will contain the original amount.

5.5.35. vic_tx_id

Name : VIC Transaction Identification
Format : 8x
Values : $[00000001 .. FFFFFFFF]_H$

Definition : This is the unique identifier of the transaction, the value is issued by the External Device and should be different for each new vmc_debit_request.
Upon receiving the pdv_debit_result, the External Device should check that the value received is an echo of the value sent in the vmc_debit_request

[4] Or <tx>vic_tx_amt

5.5.36. vic_version

Name : VIC version
Format : 2i
Values : Any value

Description : The field vic_version indicates which release version of the VIC version the ECR and the terminal use.
E.g. the value for this version is 13.

5.5.37. vmc_type (only relevant if PDV is XENTEO or XENOA)

Name : Vending machine type
Format : 6x
Values : **Bytes 1 and 2: Vendor/Machine**
Byte 3: Version number

Those values are available in the vmc_type list document. [VIC_vmc_type]

5.6. Messages contents summary

Nr	Field	Format	Max length For (ll)llvar fields[5]	vmc_purse_request	pdv_purse_result	vmc_debit_request	pdv_debit_result	vmc_cancel	pdv_error	vmc_command	pdv_command_reply	vmc_state_request	pdv_state_info	vmc_data	pdv_data
-1	vic_msg_code	2a		VP (x56 x50)	PP (x50 x50)	VD (x56 x44)	PD (X50 x44)	VC (x56 x43)	PE (x50 x45)	VM (x56 x4D)	PM (x50 x4D)	VS (x56 x53)	PS (x50 x53)	VO (x56 x4F)	PO (x50 x4F)
1	vic_bit_map	64b+		m	m	m	m	m	m	m	m	m	m	m	m
2	term_id	8a					m								
3	vic_tx_amt	6x		o		m	m								
4	iep_tx_inc	4i			m		m		m		m		m		
5	lg_cust	1i			m		m								
7	pdv_clct_nr	4i					o								
8	pdv_gen_tx_nb	8x					o								
9	pdv_state	32b			m		m		m				m		
10	pur_bal	6x			m										
11	card_id_disp	24a			m		m								
12	tx_type	2x				m									
13	sam_gen_tot	8x					o								
16	vic_card_ind	1i		m		m									
17	vic_cmd_code	4x								m					
18	vic_config_clct_amt	8x								o					
19	pdv_state_mask	32b								o					
20	vic_to	4x		m		m									
21	vic_tx_id	8x				m	m								
22	vic_msg_type	8b					m		m				m		
23	curcy	4i		o	m	m	m			o					
24	vic_to_next_request	4x		o											
25	vic_cust_ind	1i				m									
31	vmc_type	6x						m							
35	vic_proprietary_data	lllllvar	999			o	o								
36	vic_data	lllllvar	999			o	o							m	m
125	<tx>vic_tx_amt	6x					m								
126	<tx>curcy	4i					m								
141	vic_pos_entry_mode	2i			o										
142	iso2_track	38x			o	o									
143	operator_nr	4i				m									
144	function_cc	2a				o									
145	ticket_data	lllllvar	999				o				o				
146	ecr_nr	2i				o									
157	counter_tab_pa	lllllvar	n/a								o				
159	display_data	lllllvar	n/a				o						o		
167	acquirer_id	8i					o			o	o				
169	additional_amount	6x					o								
170	transaction_protocol	4x					o								
171	vic_bit_map_application_id	128b		o	m	o	m			o	o			m	m
172	transaction_identifier	8i					o								
173	date_and_time	14i					o						o		
174	brand_id	4x					o								
176	cvm	4x					o								
177	cvc2	4i				o									
178	brand_name	llvar	16				o								
179	vic_version	2i		m	m	m	m	m	o	m	m	m	m	m	m
180	card_token	lllllvar	n/a		o		o								
181	additional_info_bit_map	32b		o		o									
182	display_text_ref	4x					o						o		
183	tsc_id	12x					o								

[5] n/a indicates that no max length is specified for this field, meaning that, theoretically, the max length could be of 99 for a llvar or 9999 for a lllllvar; however the global message size must fit in the payload of a KISS frame (max. 2kB), resulting in an implied max length, depending on the presence of optional fields

6. LINE PROTOCOL : KISS

The KISS protocol is a serial byte oriented protocol, totally full duplex, asynchronous, and with no notion of master or slave.

It uses ASCII coding, and provides transparency over data field, by stuffing some characters (see Chapter Byte Stuffing).

The connection is point-to-point.

The messages are wrapped up with a header and a tail to determine the beginning, the numbering, the end and the validity. A message has a maximum length of 2Kbytes.

A message required to be sent by the application, can be repeated up to 3 times (thus a total of 4 transmissions) by the protocol when the message is not acknowledged.

If messages are lost or cannot be acknowledged, the protocol will re-synchronise itself on the numbering of the first next correctly received message.

6.1. Frame layout

There are two possible kinds of frames, which may be exchanged:

- information frames
- supervision frames

Information frames

Information frames consist of user data with a header, a trailer and a CRC (16-bit Cyclic Redundancy Check). They have the following structure:

STX	Sequence number	User data	ETX	CRC(lsb)	CRC(msb)

Supervision frames

1. Affirmative acknowledge frame

ACK	Sequence number

The Sequence Number is the number of the last frame, which has been received correctly.

2. Negative acknowledge frame

NAK

3. Flow regulation frame (Wait for ACK)

WACK

All fields are described in detail in the next sections.

6.2. Control characters

STX (0x02) Start of Text

Definition

The transmission control character STX precedes an information frame.

Description of use

1. An information frame must be preceded by STX.

2. STX resets the CRC computation to zero, when preceding an information frame.

3. If a User Data contains a character STX, it must be preceded by a DLE character (Bytes stuffing).

4. If STX appears in the sequence number, no stuffing is done.

5. STX is not included in the CRC computation.

ETX (0x03) End of Text

Definition

A transmission control character, which terminates an information frame.

Description of use

1. ETX indicates the end of an information frame.

2. ETX calls for a reply ACK, NAK or WACK from the other party.

3. ETX signals that the next following 16 bits are the CRC characters.

4. ETX is included in the CRC computation.

5. If a text contains a character ETX, it must be preceded by a DLE character (byte stuffing).

6. If ETX appears in the sequence number, no stuffing is done.

DLE (0x10) Data Link Escape

Definition

A transmission control character stuffs another control character in the data of an information frame.

Description of use

1. DLE informs the receiving station that the next character is to be treated as data and must not be considered as a control character.

2. DLE must precede STX, ETX and DLE when they are used in the data field of a message.

3. DLE is included in the CRC computation.

4. If DLE appears in the sequence number, no stuffing is done.

ACK (0x06) Affirmative Acknowledgement

Definition

Transmission control character sent by a station as an affirmative response to the other station.

Description of use

1. ACK is transmitted by one station as an affirmative response to the other station.

2. ACK is used to indicate that the last transmitted information frame has been correctly received.

3. ACK is followed by a sequence number, which is identical to the number of the information frame to which an affirmative acknowledgement is given.

NAK (0x15) Negative Acknowledgement

Definition

A transmission control character sent by a station as a negative response to the other station.

Description of use

1. NAK indicates that the last transmitted information frame was not received correctly, and the receiving station is ready to receive the same one.

2. NAK is not followed by a sequence number.

WACK (0x13) Wait for ACK (flow regulation)

Definition

A transmission control character sent by a station as a request to the other station to wait for ACK. It is however possible that data is sent before this ACK.

Description of use

1. WACK is transmitted by one station as a request to wait for ACK

2. WACK is not followed by a sequence number.

3. WACK is used to indicate that the last frame has been correctly received but has not yet been given to the application layer, because the buffers are not empty.

4. WACK will be repeated regularly (every time the receiving station tries to give the frame to the application layer).

5. There is no limit on the number of WACKs, which might be transmitted.

6. The time between successive WACKs must be less than the time-out on the ACK reception on the transmission station.

7. A WACK retriggers the ACK time-out at the transmission station.

8. When the frame is passed to the application layer, an ACK is sent.

Sequence Number (0x00..0xff) Sequence Number

Definition

A sequence number sent by both stations as a way to mark the frames. Once the sequence number has reached FF, it restarts to 00.

Description of use

1. The Sequence Number is a one byte value, and is sent just after the STX in an information frame. The same value must be repeated in the affirmative acknowledgement.

2. The Sequence number is used to compute the CRC.

3. Each station manages its own transmission sequence numbering. It should be incremented after an Affirmative Acknowledgement or after an answer time-out.

4. The Sequence number is used to avoid that the same frame is given twice (or more) to the application.

5. Value 0xff is the only value for which the receiving station always accepts the frame regardless of its previous sequence number.

6. No stuffing is done over the sequence number.

7. Upon reaching the value 0xFF, the sequence numbering should start at 0x00 again.

6.3. CRC computation

The CRC is computed over the sequence number and the ETX of a frame. It uses the polynomial $x^{15} + x^{13} + 1$, and is sent after the ETX closing the frame.

A frame is made of:

STX - Sequence number - data field - ETX - CRC(lsb) - CRC(msb)

Upon reception of the STX, the lsb and msb of the CRC field are initialised with zeroes.

For each data byte between the Sequence Number (included) until and included the closing ETX, the following algorithm is applied:

table_index = CRC(lsb) XOR data_byte
CRC(lsb) = CRC(msb) XOR lsb(table(table_index))
CRC(msb) = 0 XOR msb(table(table_index))

where :

lsb stands for Least Significant Byte
msb stands for Most Significant Byte
XOR stands for eXclusive OR
table is an array of 256 values (Refer to next page)

CRC computation table

```
0x0,      0xc0c1,   0xc181,   0x140,    0xc301    0x3c0,
0x280,    0xc241,   0xc601,   0x6c0,    0x780,    0xc741,
0x500,    0xc5c1,   0xc481,   0x440,    0xcc01,   0xcc0,
0xd80,    0xcd41,   0xf00,    0xcfc1,   0xce81,   0xe40,
0xa00,    0xcac1,   0xcb81,   0xb40,    0xc901,   0x9c0,
0x880,    0xc841,   0xd801,   0x18c0,   0x1980,   0xd941,
0x1b00,   0xdbc1,   0xda81,   0x1a40,   0x1e00,   0xdec1,
0xdf81,   0x1f40,   0xdd01,   0x1dc0,   0x1c80,   0xdc41,
0x1400,   0xd4c1,   0xd581,   0x1540,   0xd701,   0x17c0,
0x1680,   0xd641,   0xd201,   0x12c0,   0x1380,   0xd341,
0x1100,   0xd1c1,   0xd081,   0x1040,   0xf001,   0x30c0,
0x3180,   0xf141,   0x3300,   0xf3c1,   0xf281,   0x3240,
0x3600,   0xf6c1,   0xf781,   0x3740,   0xf501,   0x35c0,
0x3480,   0xf441,   0x3c00,   0xfcc1,   0xfd81,   0x3d40,
0xff01,   0x3fc0,   0x3e80,   0xfe41,   0xfa01,   0x3ac0,
0x3b80,   0xfb41,   0x3900,   0xf9c1,   0xf881,   0x3840,
0x2800,   0xe8c1,   0xe981,   0x2940,   0xeb01,   0x2bc0,
0x2a80,   0xea41,   0xee01,   0x2ec0,   0x2f80,   0xef41,
0x2d00,   0xedc1,   0xec81,   0x2c40,   0xe401,   0x24c0,
0x2580,   0xe541,   0x2700,   0xe7c1,   0xe681,   0x2640,
0x2200,   0xe2c1,   0xe381,   0x2340,   0xe101,   0x21c0,
0x2080,   0xe041,   0xa001,   0x60c0,   0x6180,   0xa141,
0x6300,   0xa3c1,   0xa281,   0x6240,   0x6600,   0xa6c1,
0xa781,   0x6740,   0xa501,   0x65c0,   0x6480,   0xa441,
0x6c00,   0xacc1,   0xad81,   0x6d40,   0xaf01,   0x6fc0,
0x6e80,   0xae41,   0xaa01,   0x6ac0,   0x6b80,   0xab41,
0x6900,   0xa9c1,   0xa881,   0x6840,   0x7800,   0xb8c1,
0xb981,   0x7940,   0xbb01,   0x7bc0,   0x7a80,   0xba41,
0xbe01,   0x7ec0,   0x7f80,   0xbf41,   0x7d00,   0xbdc1,
0xbc81,   0x7c40,   0xb401,   0x74c0,   0x7580,   0xb541,
0x7700,   0xb7c1,   0xb681,   0x7640,   0x7200,   0xb2c1,
0xb381,   0x7340,   0xb101,   0x71c0,   0x7080,   0xb041,
0x5000,   0x90c1,   0x9181,   0x5140,   0x9301,   0x53c0,
0x5280,   0x9241,   0x9601,   0x56c0,   0x5780,   0x9741,
0x5500,   0x95c1,   0x9481,   0x5440,   0x9c01,   0x5cc0,
0x5d80,   0x9d41,   0x5f00,   0x9fc1,   0x9e81,   0x5e40,
0x5a00,   0x9ac1,   0x9b81,   0x5b40,   0x9901,   0x59c0,
0x5880,   0x9841,   0x8801,   0x48c0,   0x4980,   0x8941,
0x4b00,   0x8bc1,   0x8a81,   0x4a40,   0x4e00,   0x8ec1,
0x8f81,   0x4f40,   0x8d01,   0x4dc0,   0x4c80,   0x8c41,
0x4400,   0x84c1,   0x8581,   0x4540,   0x8701,   0x47c0,
0x4680,   0x8641,   0x8201,   0x42c0,   0x4380,   0x8341,
0x4100,   0x81c1,   0x8081,   0x4040
```

6.4. Time out controls

Answer time-out

Implemented in every station.

The answer time-out is started at the transmitting station when a frame is sent out.

The answer time-out is re-triggered at the transmitting station when a WACK is received.

The answer time-out is stopped at the transmitting station when an answer (ACK, NAK) is received from the other station. Please remark that the affirmative acknowledge must be followed by the same sequence number as the frame just sent, to be considered as a valid affirmative acknowledgement.

The value of the answer time-out is computed as:
((frame length *10) /baudrate) * 2

The frame length includes the STX, sequence number, data field, ETX and CRC. This gives an extra overhead of five bytes with regard to the data field length.

Example:
For a frame length of 255 bytes, the answer time-out is **531 msec.**

Your implementation guide may contain less strict or more practical values.

Inter character time-out

Implemented in every station.

The inter character time-out prevents a station having received a STX and some more characters, to stay in this state infinitely.

The value of the inter character time-out is: 200ms.

Your implementation guide may contain less strict or more practical values.

6.5. Retry counter

Implemented in every station.

The retry counter is the maximum number of times a message may be re-transmitted in case of NAK response, or no answer at all.

Value = 3

Your implementation guide may contain less strict or more practical values.

6.6. Byte stuffing

The User data field allows transparent transmission. This means that every byte in this field may have a valid value between 0 and 255. To avoid confusion between some values and control characters, a byte stuffing mechanism is implemented at both stations.

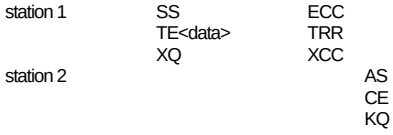
The transmitting station scans every byte it received from its application, to find a STX, a ETX or a DLE control character.
If it does, it stuffs this control character by adding a DLE just before this character.

On the receiving station, once the leading STX, and the Sequence Number are received, every DLE will be discarded, reconstituting by the same way the original User data.

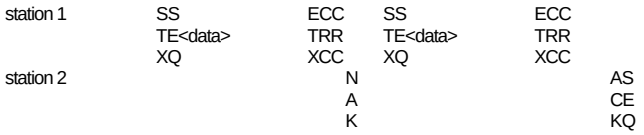
The DLEs added by this stuffing mechanism, are used in the CRC computation.

6.7. Transmission Control Sequences

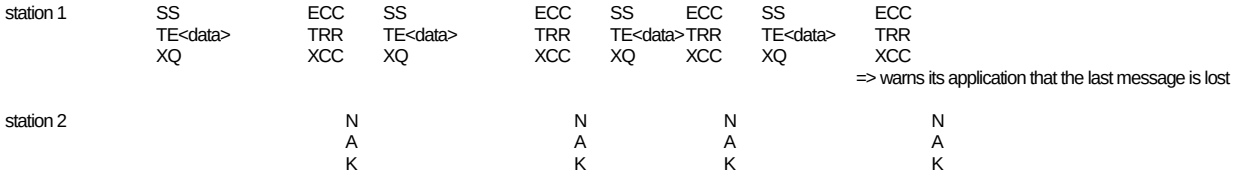
Normal message transmission



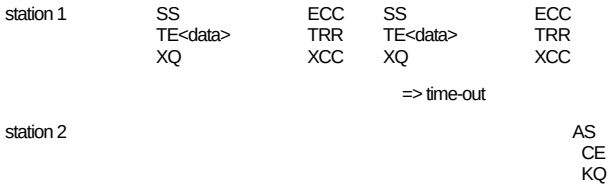
CRC error and re-transmission accepted



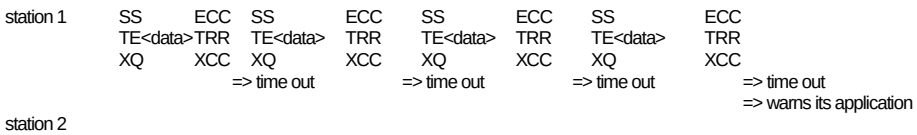
CRC error and re-transmissions refused



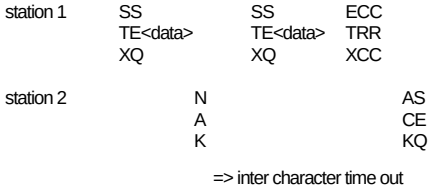
Answer time-out and re-transmission accepted



Answer time-out and re-transmissions refused



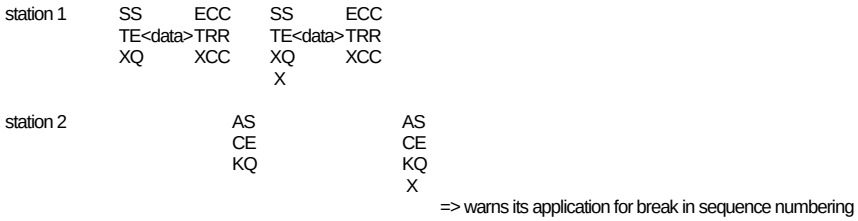
Inter character time-out and re-transmission accepted



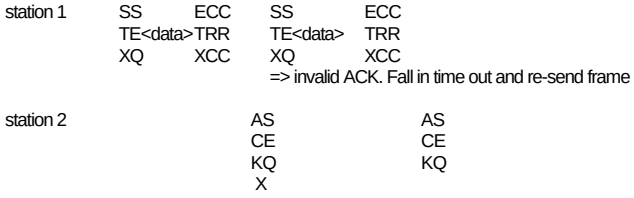
Normal full duplex exchange



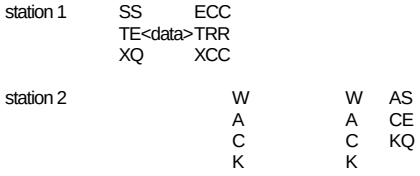
Information frame with out of order sequence number



ACK frame with out of order sequence number

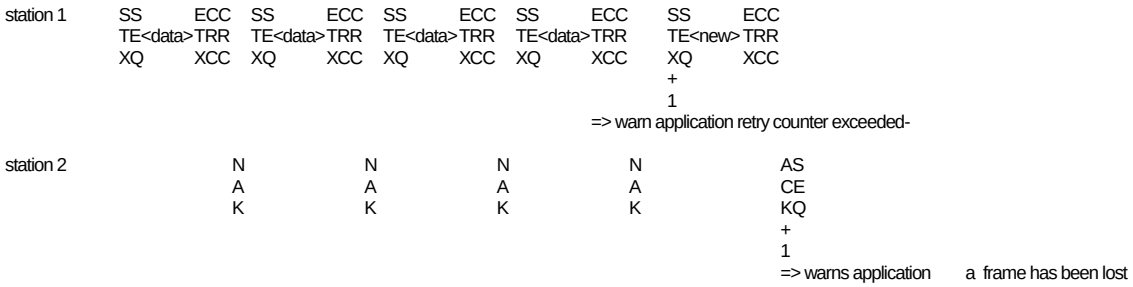


WACK frame followed by an ACK

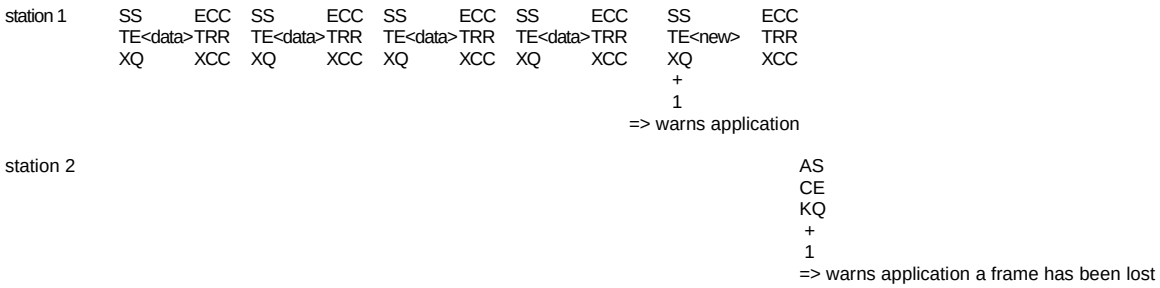


6.8. Error recovery

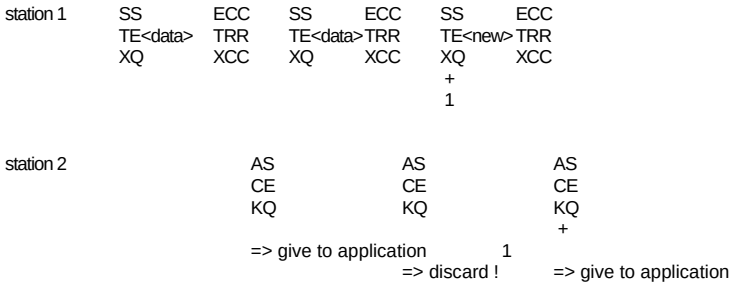
A negative (NAK) reply to an information frame causes re-transmission of the information frame. The maximal number of re-transmissions is equal to the message retry counter (see RETRY COUNTER). This implies that the information frame will be sent a number of times equal to the retry counter plus one !
When the retry counter overflows, the sending station generates an alarm to its application layer, and takes the next information frame. The sequence number must be incremented for this new information frame.



An answer time-out on an information frame on the side of the sending station, causes the re-transmission of the information frame at maximum a number of times equal to the value of the retry counter + 1. In case of unsuccessful re-sending, the sending station generates an alarm to its application layer, and take the next information frame. The sequence number must be incremented for this new information frame.



An information frame having the same Sequence Number as the previous received information frame, must be acknowledged but discarded towards its application layer.



An inter character time-out may be treated in two different ways. The easiest way is just not to answer and wait for the answer time-out to elapse, causing the re-transmission of the information frame.
A more efficient way to handle the inter character time-out is to generate a negative (NAK) reply, causing a faster re-transmission of the information frame.
The final choice is left open for the designer.

6.9. Flow chart

Definitions

ICT: inter character time-out
AT: answer time-out

ACK: ASCII character 0x06
NAK: ASCII character 0x15
WACK: ASCII character 0x13
DLE: ASCII character 0x10
STX: ASCII character 0x02
ETX: ASCII character 0x03
*: any character except ACK, NAK, WACK, DLE, STX, ETX

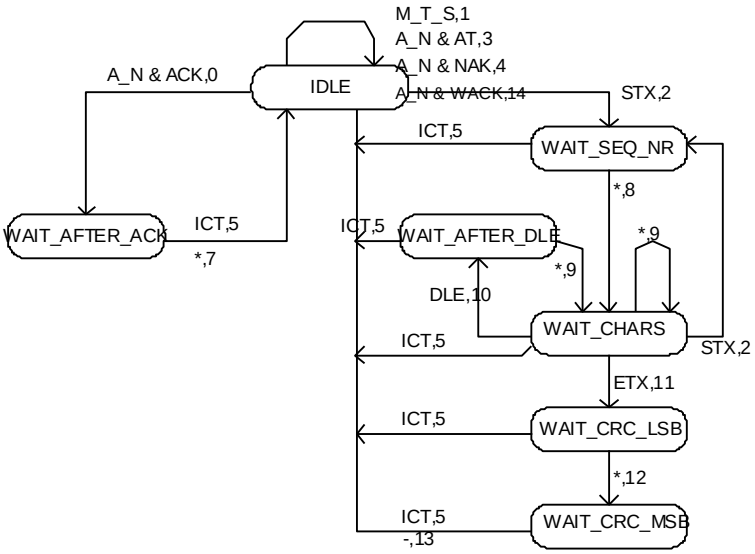
A_N: Answer_Needed : boolean specifying that the procedure waits for an answer
M_T_S: Message_To_Send : boolean specifying that the application wants to send a message
Conventions

Every state transition is characterised by an event and an action.

In the further state transition diagram, they are marked as "event, action"

Those actions in parentheses are optional.

State transition



Action list

0. do nothing

1. prepare message
send message
set A_N
start AT
reset retry_number

2. start ICT
initialise receive buffer

3. (increment time-out statistics)
action 3bis

3bis. if retry_number <= 2
send again
increment retry_number
start AT
else
signal error to application
increment transmission sequence number

4. (increment ACK statistics)
action 3 bis

5. (increment time-out statistics)

7. stop ICT
if char = transmission sequence number
stop AT
increment transmission sequence number
reset A_N
reset M_T_S
signal transmission OK to application

8. start ICT
set receive sequence number to char
compute CRC

9. stock char
compute CRC
start ICT

10. start ICT
compute CRC

11. start ICT
compute CRC
finalise CRC

12. stock char in CRC(lsb)
start ICT

13. stock char in CRC(msb)
stop ICT
if CRC <> computed CRC
send NAK
(increment NAK statistics)
if A_N then start AT
else
if receive sequence number = last receive sequence number
send ACK + receive sequence number
if A_N then start AT
else
give message to application
send ACK + receive sequence number
set last receive sequence number = receive sequence number

14. restart AT

6.10. Line characteristics

- Asynchronous
- Full duplex
- 1 start bit, 8 data bits, 1 stop bit
- Parity : None
- Speed : 9600 bps
- Physical encoding method : Non Return to Zero (NRZ)
- Byte serialisation : Least Significant Bit (LSB) first